

Roll No. _____ Subject OS Lab School/College IBF2UCS206

Sl. No.	Date	Title	Page No.	Teacher Sign / Remarks
01	27/2/26	week-0		
02	6/3/26	week-1 ↳ FCFS ↳ SJF	10	Ans 3
03	13/3/26	Week-2 - Priority		Ans 3
04	27/3/26	Week-3 ↳ Round Robin ↳ Multi-level queue		Ans 3
05	10/4/26	Week-4 ↳ RMS, EDF, proportional		Ans 3
06	24/4/26	Week-5 ↳ PC & Dining	10	Ans 3
07	8/5/26	Week-6 ↳ Deadlock Detection ↳ Banker's Algorithm		Ans 3
08	15/5/26	Week-7 ↳ Best, Worst, first Fit	10	Ans 3
09	22/5/26	Week-8 ↳ FIFO, LRU, optimal		


```
int sum=0;
for(int i=0; i<arr.length; i++)
{
    for(int j=0; j<arr.length; j++)
    {
        if(i==j)
        {
            sum+=arr[i][j];
        }
    }
    printf("%d", sum);
    return 0;
}
```

③ WAP to add rows & columns of matrix :-

Code:-

```
int main()
{
    int a[10][10];
    int r, c;
    printf("Enter no of rows & columns");
    scanf("%d %d", &r, &c);
    printf("Enter matrix elements");
    for(i=0; i<r; i++)
    {
        for(j=0; j<c; j++)
        {
            scanf("%d", &a[i][j]);
        }
    }
    printf("Sum of rows");
    for(i=0; i<r; i++)
    {
        rs=0;
        for(j=0; j<c; j++)
        {
            rs+=a[i][j];
        }
        printf("Row [%d] = ", i, rs);
    }
}
```

```
printf("Sum of Column");
for(j=0; j<c; j++)
{
    cs=0;
    for(i=0; i<r; i++)
    {
        cs+=a[i][j];
    }
    printf("Column [%d] = %d", j+1, cs);
}
return 0;
}
```

④ Program to count total no of duplicate no of elements in array

Code:-

```
int main()
{
    int a[100], n, i, j, count=0;
    printf("Enter no of elements");
    scanf("%d", &n);
    printf("Enter elements");
    for(i=0; i<n; i++)
    {
        scanf("%d", &a[i]);
    }
    for(i=0; i<n; i++)
    {
        for(j=i+1; j<n; j++)
        {
            if(a[i]==a[j])
            {
                count++;
                break;
            }
        }
    }
    printf("Total duplicate elements = %d", count);
    return 0;
}
```

⑤ WAP to find 2nd Largest element in an array.

```
code:- int main() {
    int a[100], n, i;
    int largest, second;
    printf("Enter elements ");
    for(i=0; i<n; i++) {
        scanf("%d", &a[i]);
    }
    largest = second = a[0];
    for(i=1; i<n; i++) {
        if(a[i] > largest) {
            second = largest;
            largest = a[i];
        }
        else if(a[i] > second && a[i] != largest) {
            second = a[i];
        }
    }
    printf("2nd Largest element = %d", second);
    return 0;
}
```

⑥ WAP to delete an element at a desired positⁿ in an array.

```
code:- int main() {
    int a[100], n, pos, i;
    printf("Enter elements ");
    for(i=0; i<n; i++) {
        scanf("%d", &a[i]);
    }
}
```

```
printf("Enter pos to delete ");
scanf("%d", &pos);
for(i=pos-1; i<n-1; i++) {
    a[i] = a[i+1];
}
n--;
printf("Array after deletion");
for(i=0; i<n; i++) {
    printf("%d", a[i]);
}
return 0;
}
```

~~Q. 3~~

6/3/26

Lab-1Date ___/___/___
Page _____WAP to simulate cpu scheduling
Algorithm

- ① FCFS
- ② SJF

① FCFS:

```
code:- #include <stdio.h>
#include <stdlib.h>
```

```
int main()
```

```
{
```

```
int n, i, choice;
```

```
int at[10], bt[10], ct[10],
tat[10], wt[10];
```

```
printf("Enter no of processes:");
```

```
scanf("%d", &n);
```

```
for(i=0; i<n; i++){
```

```
printf("Enter AT & BT of  
Process P%d", i+1); }
```

```
printf("Enter choice");
```

```
scanf("%d", &choice);
```

```
if (choice != 1)
```

```
{
```

```
ct[0] = at[0] + bt[0];
printf("Select Type: 1. Non-prem-  
-ptive, 2. Preemptive");
```

```
if (choice == 1)
```

```
{
```

```
ct[0] = at[0] + bt[0];
```

Date ___/___/___
Page _____

```
for(i=1; i<n; i++){
if(ct[i-1] < at[i])
ct[i] = at[i] + bt[i];
else
ct[i] = ct[i-1] + bt[i];
}
```

```
for(i=0; i<n; i++){
tat[i] = ct[i] - at[i];
wt[i] = tat[i] - bt[i];
}
```

```
printf("\n P\tAT\tBT\tCT\tTAT\tWT\n");
```

```
for(i=0; i<n; i++){
printf("P%d\t\t%d\t\t%d\t\t%d\t\t",
i+1, at[i], bt[i], ct[i],
tat[i], wt[i]); }
```

```
else {
```

```
printf("FCFS does not support  
preemptive scheduling");
```

```
}
```

```
return 0;
```

```
}
```

② SJF

```

Code:- #include <stdio.h>
#include <stdlib.h>
int main() {
    int n, i, j, time = 0, choice, done = 0;
    int small;
    int at[10], bt[10], rt[10],
        ct[10], tat[10], wt[10];
    int completed = 0;
    printf("Enter number processes");
    scanf("%d", &n);
    for (i = 0; i < n; i++) {
        printf("Enter AT & BT for P%d", i+1);
        scanf("%d %d", &at[i], &bt[i]);
        rt[i] = bt[i];
    }
    printf("Select\n1. Non-preemptive,\n2. Preemptive");
    scanf("%d", &choice);
    if (choice == 1) {
        int vis[10] = {0};
        while (done < n) {
            int min = INT_MAX;
            int p = -1;
            for (i = 0; i < n; i++) {
                if (at[i] <= time &&
                    (vis[i] && bt[i] < min)) {
                    min = bt[i]; p = i;
                }
            }
            if (p == -1) {
                time++;
                continue;
            }
            rt[p]--;
            time++;
            if (rt[p] == 0) {
                ct[p] = time;
                done++;
            }
        }
    }
}

```

```

min = bt[i];
p = i;
if (p == -1) {
    time++;
    continue;
}
time += bt[p];
ct[p] = time;
vis[p] = 1;
done++;
}
else {
    while (done < n) {
        int min = INT_MAX;
        p = -1;
        for (i = 0; i < n; i++) {
            if (at[i] <= time &&
                (rt[i] > 0 &&
                 rt[i] < min)) {
                min = rt[i]; p = i;
            }
        }
        if (p == -1) {
            time++;
            continue;
        }
        rt[p]--;
        time++;
        if (rt[p] == 0) {
            ct[p] = time;
            done++;
        }
    }
}
}

```

```
printf("In P | AT | BT | CT | TAT\n");
printf("WT\n");
for(i=0; i<n; i++) {
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    printf("P%d | AT%d | BT%d | CT%d | TAT",
        i+1, at[i], bt[i], ct[i],
        tat[i], wt[i]);
}
return 0;
```

① FCFS output:-

Enter number of process: 5

Enter AT & BT for P1: 0 2

Enter AT & BT for P2: 1 4

Enter AT & BT for P3: 2 5

Enter AT & BT for P4: 3 1

Enter AT & BT for P5: 4 2

P	AT	BT	CT	TAT	WT
P1	0	2	2	2	0
P2	1	4	6	5	1
P3	2	5	11	9	4
P4	3	1	12	9	8
P5	4	2	14	10	8

② SJF output:-

Enter no of processes: 4

AT and BT of P1: 0 2

P2 0 3

P3 1 4

P4 3 1

Select Type

① Non-preemptive

② Preemptive

Enter choice: 1

P	AT	BT	CT	TAT	WT
P1	0	2	2	2	0
P2	2	3	5	3	0
P3	1	4	10	9	5
P4	3	1	6	3	2

Average TAT: 17.000, WT: 7.0000

Enter choice: 2

P	AT	BT	CT	TAT	WT
P1	0	2	2	2	0
P2	1	3	6	5	2
P3	3	1	4	1	0
P4	4	5	11	7	2

Average TAT: 15.0000, WT: 4.0000

Conclusion:- Preemptive is better

13/3/26

Date ___/___/___
Page _____Week-2

- ① WACP to simulate the following CPU scheduling algorithm to find TAT and WT
 ↳ priority (pre-emptive & non-preemptive)

Code:-

```
#include <stdio.h>
int main() {
    int n, i, choice;
    printf("Enter number of processes:");
    scanf("%d", &n);
    int at[n], bt[n], rt[n], pr[n],
    et[n], wt[n], tat[n];
    for(i=0; i<n; i++) {
        printf("Enter AT, BT, Priority\nof P%d: ", i+1);
        scanf("%d%d%d", &at[i],
        &bt[i], &pr[i]);
        rt[i] = bt[i];
        printf("1. Non Preemptive Priority");
        printf("2. Pre-emptive Priority");
        printf("Enter choice");
        scanf("%d", &choice);
        int time = 0, count = 0;
```

Date ___/___/___
Page _____

```
if(choice == 1) {
    int completed[n];
    for(i=0; i<n; i++) completed[i] = 0;
    while(count < n) {
        int idx = -1;
        int highest = 9999;
        for(i=0; i<n; i++) {
            if(at[i] <= time && completed[i] == 0) {
                if(pr[i] < highest) {
                    highest = pr[i];
                    idx = i;
                }
            }
        }
        if(idx != -1) {
            time += bt[idx];
            ct[idx] = time;
            tat[idx] = ct[idx] - at[idx];
            wt[idx] = tat[idx] - bt[idx];
            completed[idx] = 1;
            count++;
        } else {
            time++;
        }
    }
} else if(choice == 2) {
    while(count < n) {
        int idx = -1;
        int highest = 9999;
```

```

if (id[x] == -1) {
    rt[id[x]] = j;
    time++;
    if (rt[id[x]] == 0) {
        ct[id[x]] = time;
        tat[id[x]] = ct[id[x]] -
            at[id[x]];
        wt[id[x]] = tat[id[x]] -
            bt[id[x]];
        count++;
    }
}

```

Output:-

Enter number of processes : 3
Enter AT, BT and Priority of P1: 0 2 1
P2: 0 6 3
P3: 4 2 2

1. Non-Preemptive Priority
2. Preemptive Priority

P	AT	BT	PR	CT	TAT	WT
P1	0	2	1	2	2	0
P2	0	6	3	8	8	2
P3	4	2	2	10	6	4

Average TAT: 5.333, WT: 2.000

Enter choice: 2

P	AT	BT	PR	CT	TAT	WT
P1	0	2	1	2	2	0
P2	0	6	3	10	10	4
P3	4	2	2	2	2	0

Average TAT: 4.6667, WT: 1.3333

27/3/26

Week-3

Date ___/___/___
Page ___

① ↳ WACP to execute Round Robin with different quantum sizes.

Code:-

```
#include <stdio.h>
#define Max 100

int main() {
    int n, q-count;
    int a[Max], b[Max],
        r[Max], c[Max], tat[Max],
        wt[Max], queue[Max],
        visited[Max];
    int front, rear, time, completed;
    printf("Enter number of processes:");
    scanf("%d", &n);
    printf("Enter AT & BT: \n");
    for (int i=0; i<n; i++) {
        printf("P%d AT BT:", i+1);
        scanf("%d %d", &a[i], &b[i]);
    }
    printf("Enter no of quantum values");
    scanf("%d", &q-count);
    int quantum [q-count];
    printf("Enter quantum values: \n");
    for (int i=0; i<q-count; i++) {
        for (int j=0; j<n; j++) {
            r[j] = b[j];
            visited[j] = 0;
        }
    }
```

Date ___/___/___
Page ___

```
front = rear = 0;
time = 0;
completed = 0;
queue[rear++] = 0;
visited[0] = 1;
while (completed < n) {
    if (front == rear)
        break;
    int i = queue[front++];
    if (r[i] > quantum[q]) {
        time += quantum[q];
        r[i] -= quantum[q];
    }
    else {
        time += r[i];
        r[i] = 0;
        c[i] = time;
        completed++;
    }
    for (int j=0; j<n; j++) {
        if (!visited[j] && a[j] <= time) {
            queue[rear++] = j;
            visited[j] = 1;
        }
        if (r[j] > 0) {
            queue[rear++] = j;
        }
        if (front == rear) {
            for (int j=0; j<n; j++) {
                if (r[j] > 0) {
                    queue[rear++] = j;
                    visited[j] = 1;
                    time = a[j];
                    break;
                }
            }
        }
        for (int i=0; i<n; i++) {
            tat[i] = c[i] - a[i];
            wt[i] = tat[i] - b[i];
        }
    }
```

```
total_tat += tat[i];
total_wt += wt[i];
printf("P%d\t\t%d\t\t%d\t\t%d\t\t%d\n",
       i+1, at[i], bt[i], ct[i],
       tat[i], wt[i]);
printf("Average TAT= %f\n", total_tat/n);
printf("Average WT= %f\n", total_wt/n);
return 0;
```

y

Output:-

Enter number of Processes: 3
Enter AT and BT:
P1 AT BT: 0 7
P2 AT BT: 3 5
P3 AT BT: 4 6
Enter no of Quantum values: 2
Enter quantum values:
4
6

Quantum = 4

PID	AT	BT	CT	TAT	WT
P1	0	7	15	15	8
P2	3	5	16	13	8
P3	4	6	18	14	8

Avg TAT = 14.00

Avg WT = 8.00

Quantum = 6

PID	AT	BT	CT	TAT	WT
P1	0	7	18	18	11
P2	3	5	11	8	3
P3	4	6	17	13	7

Avg TAT = 13.00

Avg WT = 7.00

∴ Quantum Time is more than TAT & WT is less

② Multi Level Queue:-

Code: #include <stdio.h>
#include <string.h>
int main()

int n;

printf("Enter no of processes:");

scanf("%d", &n);

int pid[n], at[n], bt[n], rt[n];

for(int i=0; i<n; i++)

{ printf("P%d, AT, BT", i+1, &pid[i], &at[i], &bt[i]);

printf("Enter Type (system/user):");

scanf("%s", type[i]);

rt[i] = bt[i];

y

for(int i=0; i<n-1; i++)

{ for(int j=0; j<n-i-1; j++)

{ if (at[j] > at[j+1])

{ int temp;

temp = at[j]; at[j] = at[j+1];

at[j+1] = temp;

temp = bt[j]; bt[j] = bt[j+1];

bt[j+1] = temp;

```
temp = type[j]; type[j] = type[j+1];
type[j+1] = temp;
y y y
```

```
int system[100], user[100];
int sf=0, sr=0, vf=0, ur=0;
int current_time=0,
    completed=0, i=0;
int current=-1;
while (completed < n) {
    while (i < n && at[i] <= current_time)
        if (type[i] == 0)
            system[sf++] = i;
        else
            user[ur++] = i;
        i++;
    if (current != -1) {
        if (type[current] == -1 && sf < sr) {
            user[ur++] = current;
            current = -1;
        }
        if (current == -1) {
            if (sf < sr)
                current = system[sf++];
            else if (uf < ur)
                current = user[ur++];
            else {
                current_time++;
                continue;
            }
        }
    }
}
```

```
for (int i=0; i < n; i++) {
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];
    total_wt += wt[i];
    total_tat += tat[i];
    printf("\n ID \t Type \t AT \t BT \t CT \t WT \t TAT\n");
    for (int i=0; i < n; i++) {
        if (type[i] == 0)
            printf("%d \t System \t", pid[i]);
        else
            printf("%d \t User \t", pid[i]);
        printf("%d \t %d \t %d \t %d \t %d\n",
            at[i], bt[i], ct[i], wt[i],
            tat[i]);
    }
    printf("\n Avg TAT %f", total_tat/n);
    printf("\n Avg WT %f", total_wt/n);
    return 0;
}
```

Output:-

Enter number of processes: 4

Process 1

AT and BT: 0 3

0 (or) 1: 0

Process 2

AT and BT: 2 5

0 (or) 1: 1

Process 3

AT and BT: 3 7

0 (or) 1: 0

Process 4

AT and BT: 3 5

0 (or) 1: 0

ID	Type	AT	BT	CT	WT	TAT
1	System	0	3	3	0	3
2	User	2	5	18	11	16
3	System	3	7	10	0	7
4	System	5	3	13	5	8

Avg TAT: 4.00

Avg WT: 8.50

10/4/26

Week-4

- ① Write C program to simulate Real-Time CPU Scheduling algorithm
- Rate - Monotic
 - Earliest-deadline first
 - proportional Scheduling

Code:-

```
#include <stdio.h>
#include <math.h>
#define MAX 10
```

```
typedef struct {
    int id;
    int bt;
    int deadline;
    int period;
    int ct, wt, tat;
} Process;
```

```
Process p[MAX], temp;
int n;
```

```
float calculatedUtilization(EFC) {
    float util = 0;
    for (int i = 0; i < n; i++) {
        util += (float) p[i].bt / p[i].
            deadline;
    }
    return util;
}
```

```
float calculateUtilizationRMS() {
    float util = 0;
    float
    for (int i = 0; i < n; i++) {
        util += (float) p[i].bt /
                p[i].period;
    }
    return util util;
}
```

```
void edfScheduling() {
    for (int i = 0; i < n; i++) {
        for (int j = i + 1; j < n; j++) {
            if (p[i].deadline > p[j].deadline) {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
    int time = 0;
    for (int i = 0; i < n; i++) {
        time += p[i].bt;
        p[i].ct = time;
        p[i].tat = p[i].ct;
        p[i].wt = p[i].tat -
                p[i].bt;
    }
}
```

```
float util = calculateUtilizationEDF();
printf("\n == Earliest
Deadline first (EDF)
Scheduling == \n");
```

```
printf("CPU Utilization : %.2f\n",
        util);
if (util <= 1)
    printf("Schedulable Utilization
    <= 1\n");
else
    printf("Not Schedulable\n");

printf("ID BT Deadline CT WT
    TAT\n");
for (int i = 0; i < n; i++) {
    printf("%d %d %d %d\n",
        p[i].id, p[i].bt, p[i].deadline,
        p[i].ct, p[i].wt, p[i].tat);
}
```

```
void rmsScheduling() {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if (p[i].period > p[j].period) {
                temp = p[i];
                p[i] = p[j];
                p[j] = temp;
            }
        }
    }
}
```

```
int time = 0;
for (int i = 0; i < n; i++) {
    time += p[i].bt;
    p[i].ct = time;
    p[i].tat = p[i].ct;
    p[i].wt = p[i].tat - p[i].bt;
}
```

```
float util = calculateUtilization(RMS);
float bound = n * (pow(2, (1.0/n)) - 1);
printf("\n=== Rate Monotonic  
Scheduling (RMS) ===\n");
printf("CPU Utilization: %.2f/n",
    util);
printf("RM Bound: %.4f/n",
    bound);
if (util <= bound)
    printf("Schedulable  
(Utilization <= RM bound)\n");
else
    printf("Not Schedulable\n");

printf("TID\tBT\tPeriod\t  
CT\tWT\tWT\n");
for (int i=0; i<n; i++) {
    printf("%d\t%d\t%d\t%d\t",
        p[i].id, p[i].bt, p[i].period,
        p[i].ct, p[i].wt, p[i].tt);
}

int main() {
    printf("Enter number of  
processes: ");
    scanf("%d", &n);
    printf("\nEnter process  
details:\n");
```

```
for (int i=0; i<n; i++) {
    p[i].id = i;
    printf("Process %d:\n", i);
    printf("Burst Time: ");
    scanf("%d", &p[i].bt);
    printf("Deadline (for EDF): ");
    scanf("%d", &p[i].deadline);
    printf("Period (for RMS): ");
    scanf("%d", &p[i].period);
}
edfScheduling();
rmsScheduling();
return 0;
}
```

Output:-

Enter number of processes: 3

process: 0
BT: 3
Deadline (for EDF): 7
period (for RMS): 20

process: 1
BT: 2
Deadline (for EDF): 4
period (for RMS): 5

process: 2
BT: 2
Deadline (for EDF): 8
Period (for RMS): 10

== EDF Scheduling ==

CPU utilization: ~~0.75~~ 0.75

~~Not~~ Schedulable:

Schedulable (utilization ≤ 1)

ID	BT	D	CT	WT	TAT
0	3	7	5	2	6
1	2	4	3	0	3
2	2	9	8	6	8

== Rate Monotonic Scheduling (RMS) ==

CPU Utilization: 0.79

RM bound: 0.7891

Schedulable (utilization $\leq$ RM bound)

ID	BT	Period	CT	WT	TAT
0	3	20	8	4	7
1	1	4	3	0	5
2	2	8	7	3	3

== proportional Scheduling ==
Total weight = 6

ID	Weight	CPU share
0	2	0.33
1	1	0.17
2	3	0.50

Week-5

WACP for producer-consumer problem using semaphore

code:-

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define Buffer-Size 5
int buffer[Buffer-Size];
int in=0, out=0;
sem_t empty, full, mutex;
#define wait(x) sem_wait(&x)
#define signal(x) sem_post(&x)
void *producer(void *arg)
{
    int item;
    while(1)
    {
        item = rand() % 10;
        wait(empty);
        wait(mutex);
        buffer[in] = item;
        printf("Produced: %d at\n", item, in);
        in = (in+1) % Buffer-Size;
        signal(mutex);
        signal(full);
        sleep(1);
    }
}
```

```
void *consumer(void *arg) {
    int item;
    while (1) {
        wait(full);
        wait(mutex);
        item = buffer[out];
        out = (out + 1) % Buffer-Size;
        signal(mutex);
        signal(empty);
        sleep(2);
    }
}
```

y

```
int main() {
    pthread_t p, c;
    sem_init(&empty, 0, Buffer-Size);
    sem_init(&full, 0, 0);
    sem_init(&mutex, 0, 1);
    pthread_create(&p, NULL, producer,
                  NULL);
    pthread_create(&c, NULL, consumer,
                  NULL);
    pthread_join(p, NULL);
    pthread_join(c, NULL);
    return 0;
}
```

Output:-
 produced: 3 at 0
 consumed: 3 from 0
 produced: 6 at 1
 consumed: 6 from 1
 produced: 9 at 3
 consumed: 8 from 0

produced: 6 at 0
 consumed: 5 from 1
 produced: 3 at 1

Dining

② Dining - philosopher's -

code:-

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>
#define N 5
sem_t forks[N];
sem_t mutex;
#define wait(x) sem_wait(&x)
#define signal(x) sem_post(&x)
```

```
void *philosopher(void *num) {
    int id = *(int *)num;
    while (1) {
        printf("philosopher %d is\n", id);
        thinking\n";
        sleep(1);
        wait(mutex);
        wait(forks[id]);
        wait(forks[(id + 1) % N]);
        printf("philosopher %d is\n", id);
        eating\n";
        sleep(2);
        signal(forks[id]);
    }
}
```

```

signal(forks[id+1] % N);
signal(mutex);
}
}
int main()
{
pthread_t ph[N];
int ids[N];
sem_init(&mutex, 0, 1);
for(int i=0; i<N; i++)
    sem_init(&forks[i], 0, 1);
for(int i=0; i<N; i++) {
    ids[i] = i;
    pthread_create(&ph[i], NULL,
        philosopher, &ids[i]);
}
for(int i=0; i<N; i++)
    pthread_join(ph[i], NULL);
return 0;
}

```

Output:-

philosopher 0 is thinking
 philosopher 3 is thinking
 philosopher 4 is thinking
 philosopher 1 is thinking
 philosopher 2 is thinking
 philosopher 0 is eating
 philosopher 0 is thinking
 philosopher 3 is eating
 philosopher 3 is thinking
 philosopher 1 is eating

Tracing

① Operation Buffer state id out

opⁿ Buffer

Tracing: ① -

op ⁿ	Buffer state	in	out	empty	full	mutex
P 3 at 0	[3, -, -, -]					
C 3 from 0	[-, -, -, -]					
P 8 at 0	[8, -, -, -]					
P 6 at 1	[2, 6, -, -]					
P 9 at 2						
C 6 from 1	[-, -, -, -]					
P 9 at 3	[-, -, -, 9]					
C 8 from 0	Blocked					
P 6 at 0	[6, -, -, 9]					
C 5 from 1						

8/5/26

Week-6

Tracing :- ①

operation	Buffer State	In	Out	Empty	Full	Mutex
P: 3 at 0	[3 - - -]	1	0	3	1	1
C: 3 from 0	[- - - -]	1	1	4	0	1
P: 6 at 1	[- 6 - -]	2	1	3	1	1
P: 7 at 2	[- 6 7 -]	3	1	2	2	1
C: 6 from 1	[- - 7 -]	3	2	3	1	1
P: 5 at 3	[- - 7 5]	4	2	2	2	1
P: 3 at 0	[3 - 7 5]	1	2	1	3	1
C: 7 from 2	[3 - - 5]	1	3	2	2	1

Tracing ②

Philosopher	State Change	Forks used
P0	Think	-
P1	Think	-
P2	Think	-
P3	Think	-
P4	Think	-
P3	Eat	f3, f4
P3	Think	Released f3, f4
P4	Eat	f4, f0
P4	Think	Released f4, f0

Write a C program to simulate:-
① Banker's algorithm for the purpose of deadlock avoidance

② Deadlock Detection

① Code:-

```
#include <stdio.h>
int main() {
    int n = 5, m = 3;
    int i, j, k;
    int alloc[5][3] = {
        {0, 1, 0},
        {2, 0, 0},
        {3, 0, 2},
        {2, 1, 1},
        {0, 0, 2}};
    int max[5][3] = {
        {7, 5, 3},
        {3, 2, 2},
        {9, 0, 2},
        {2, 2, 2},
        {4, 3, 3}};
    int avail[5] = {3, 3, 2};
    int need[5][3];
    for (i = 0; i < n; i++) {
        for (j = 0; j < m; j++) {
            need[i][j] = max[i][j] -
                alloc[i][j];
        }
    }
}
```

```
printf("Need Matrix:\n");
for(i=0; i<n; i++) {
    printf("P%d: ", i);
    for(j=0; j<m; j++) {
        printf("%d", need[i][j]);
    }
    printf("\n");
}
```

```
int finish[5]=404;
int safeSeq[5];
int count=0;
while(count<n) {
    int found=0;
    for(i=0; i<n; i++) {
        if(finish[i]==0) {
            for(j=0; j<m; j++) {
                if(need[i][j]>avail[j])
                    break;
            }
            if(j==m) {
                for(k=0; k<m; k++) {
                    avail[k]+=alloc[i][k];
                }
                safeSeq[count++]=i;
                finish[i]=1;
                found=1;
            }
        }
    }
}
```

```
if(found==0) {
    printf("\n System is NOT in  
safe state\n");
    return 0;
}
```

```
printf("\n System is in SAFE  
state\n");
printf("\n Safe Sequence: \n");
for(i=0; i<n; i++) {
    printf("P%d", safeSeq[i]);
    if(i!=n-1)
        printf("→");
}
printf("\n");
return 0;
}
```

Output:-

Enter n and m values: 5 3

Enter Allocation matrix:

```
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
```

Enter max matrix:

```
7 5 3
3 2 2
7 0 2
2 2 2
4 3 3
```

Need Matrix:

```
P0: 7 4 3
P1: 1 2 2
P2: 6 0 0
P3: 0 1 1
P4: 4 3 1
```

System is in SAFE state

Safe Sequence:

$P_1 \rightarrow P_3 \rightarrow P_4 \rightarrow P_0 \rightarrow P_2$

② code:-

```
#include <stdio.h>
```

```
int main() {
```

```
    int n, m, i, j, k;
```

```
    printf("\nEnter number of processes: ");
```

```
    scanf("%d", &n);
```

```
    printf("\nEnter number of resources: ");
```

```
    scanf("%d", &m);
```

```
    int alloc[n][m], request[n][m];
```

```
    int avail[m];
```

```
    printf("\nEnter Allocation Matrix:");
```

```
    for(i=0; i<n; i++)
```

```
        for(j=0; j<m; j++)
```

```
            scanf("%d", &alloc[i][j]);
```

```
    printf("\nEnter Available Resources:");
```

```
    for(i=0; i<m; i++)
```

```
        scanf("%d", &avail[i]);
```

```
    int finish[n];
```

```
    for(i=0; i<n; i++)
```

```
        for int empty = 1;
```

```
        for(j=0; j<m; j++) {
```

```
            if(alloc[i][j] = 0) {
```

```
                empty = 0;
```

```
                break;
```

```
        finish[i] = empty;
```

```
    int found;
```

```
    do {
```

```
        found = 0;
```

```
        for(i=0; i<n; i++) {
```

```
            if(!finish[i]) {
```

```
                for(j=0; j<m; j++) {
```

```
                    if(request[i][j] > avail[j])
```

```
                        break;
```

```
                }
```

```
            if(j == m) {
```

```
                for(k=0; k<m; k++)
```

```
                    avail[k] += alloc[i][k];
```

```
                finish[i] = 1;
```

```
                found = 1;
```

```
            }
```

```
        }
```

```
        }
```

```
    while(found);
```

```
    int deadlock = 0;
```

```
    for(i=0; i<n; i++) {
```

```
        if(!finish[i]) {
```

```
            deadlock = 1;
```

```
            printf("\nProcess P%d is in\n", i);
```

```
        }
```

```
    }
```

```
    if(!deadlock)
```

```
        printf("\nNo Deadlock detected\n");
```

```
    return 0;
```

Output:-

Enter number of processes: 4

Enter number of resources: 3

Enter Allocation matrix:-

1 0 2

2 1 1

1 0 3

1 2 2

Enter Request Matrix:-

0 0 1

1 0 2

0 0 0

3 3 0

Enter Available Resources:

0 0 0

Process P3 is in deadlock

~~Not a deadlock~~

Week-7

Q1) WACP to simulate the following contiguous memory allocation technique

(a) worst-fit

(b) Best fit

(c) first-fit

Code:-

```
#include <stdio.h>
```

```
#define MAX 25
```

```
void firstfit (int blockSize[],  
              int m, int processSize[],  
              int n)
```

```
{
```

```
    int allocation [MAX];
```

```
    int temp [MAX];
```

```
    for (int i=0; i<m; i++)
```

```
        temp[i] = blockSize[i];
```

```
    for (int i=0; i<n; i++)
```

```
        allocation[i] = -1;
```

```
    for (int i=0; i<n; i++) {
```

```
        for (int j=0; j<m; j++) {
```

```
            if (temp[j] >= processSize[i]) {
```

```
                allocation[i] = j;
```

```
                temp[j] = processSize[i];
```

```
                break;
```

```
            }
```

```
        }
```

```
}
```

```
void bestfit(int blockSize[],
int m, int processSize[], int n)
```

```
{
    int allocation[MAX];
    int temp[MAX];
    for(int i=0; i<m; i++)
        temp[i] = blockSize[i];
    for(int i=0; i<n; i++)
        allocation[i] = -1;
    for(int i=0; i<n; i++) {
        int bestIdx = -1;
        for(int j=0; j<m; j++) {
            if(temp[j] >= processSize[i]) {
                if(bestIdx == -1 ||
                    temp[j] < temp[bestIdx])
                    bestIdx = j;
            }
        }
    }
}
```

```
if(bestIdx != -1) {
    allocation[i] = bestIdx;
    temp[bestIdx] -= processSize[i];
}
```

```
void worstfit(int blockSize[], int m,
int processSize[], int n) {
```

```
    int allocation[MAX];
    for(int i=0; i<m; i++)
        temp[i] = blockSize[i];
```

```
for(int i=0; i<n; i++) {
    int worstIdx = -1;
    for(int j=0; j<m; j++) {
        if(temp[j] >= processSize[i]) {
            if(worstIdx == -1 ||
                temp[j] > temp[worstIdx])
                worstIdx = j;
        }
    }
}
```

```
if(worstIdx != -1) {
    allocation[i] = worstIdx;
    temp[worstIdx] -= processSize[i];
}
```

```
int main() {
    int blockSize[MAX], processSize
    [MAX];
```

```
    int m, n;
    printf("Enter no of memory
    blocks");
    scanf("%d", &m);
    printf("Enter sizes of %d memory
    blocks");
    for(int i=0; i<m; i++)
```

```
    scanf("%d", &blockSize[i]);
}
```

```
printf("Enter size of %d\n", n);
for (int i=0; i<n; i++) {
    scanf("%d", &processSize[i]);
}
firstFit(blockSize, m, processSize, n);
bestFit(blockSize, m, processSize, n);
worstFit(blockSize, m, processSize, n);
return 0;
}
```

Output:-

Enter no of memory blocks: 5
Enter sizes of 5 memory blocks:
200 300 400 500 600

Enter no of processes: 4
Enter sizes of 4 processes:
110 120 130 140

First Fit

Process No	Process Size	Block No
1	110	1
2	120	2
3	130	2
4	140	3

Best Fit

Process No.	Process Size	Block No
1	110	1
2	120	2
3	130	2
4	140	3

Worst Fit

Process No.	Process Size	Block No
1	110	5
2	120	4
3	130	5
4	140	3

22/9/26

Week-8

Date ___/___/___
Page ___

WACP to simulate the following
page allocations Technique

- FIFO
- LRU
- Optimal

Code:-

```
#include <stdio.h>
int frames[10];
void fifo(int pages[], int n, int f)
{
    int i, j, k=0, fault=0, found;
    for(i=0; i<f; i++)
        frames[i] = -1;
    for(i=0; i<n; i++)
    {
        found = 0;
        for(j=0; j<f; j++)
        {
            if(frames[j] == pages[i])
            {
                found = 1;
                break;
            }
        }
        if(!found)
        {
            frames[k] = pages[i];
            k = (k+1) % f;
            fault++;
        }
    }
    printf("Fifo page faults = %d\n", fault);
}
```

Date ___/___/___
Page ___

```
void lru(int pages[], int n, int f)
{
    int i, j, least, pos, fault=0;
    int time[10], counter=0;
    for(i=0; i<f; i++)
    {
        frames[i] = -1;
        time[i] = 0;
    }
    for(i=0; i<n; i++)
    {
        int found = 0;
        for(j=0; j<f; j++)
        {
            if(frames[j] == pages[i])
            {
                counter++;
                time[j] = counter;
                found = 1;
                break;
            }
        }
        if(!found)
        {
            least = time[0];
            pos = 0;
            for(j=0; j<f; j++)
            {
                if(frames[j] == -1)
                {
                    pos = j;
                    break;
                }
            }
            if(time[j] < least)
            {
                least = time[j];
                pos = j;
            }
            frames[pos] = pages[i];
            counter++;
            time[pos] = counter;
        }
    }
}
```

```

    fault++;
    printf("LRU page faults = %d\n",
           fault);

```

```

void optimal(int pages[], int n, int f)
{
    int i, j, k, pos, farthest, fault=0;
    for(i=0; i<f; i++)
        frames[i] = -1;
    for(i=0; i<n; i++)
    {
        int found=0;
        for(j=0; j<f; j++)
        {
            if(frames[j] == pages[i])
            {
                found = 1;
                break;
            }
        }

```

```

        if(!found)
        {
            pos = -1;
            for(j=0; j<f; j++)
            {
                if(frames[j] == -1)
                {
                    pos = j;
                    break;
                }
            }

```

```

            if(pos == -1)
            {
                farthest = i;
                pos = 0;
                for(j=0; j<f; j++)

```

```

        int next = -1;
        for(k=i+1; k<n; k++)
        {
            if(frames[k] == pages[i])
            {
                next = k;
                break;
            }
        }

```

```

        if(next == -1)
        {
            pos = j;
            break;
        }
        if(next > farthest)
        {
            farthest = next;
            pos = j;
        }

```

```

        frames[pos] = pages[i];
        fault++;
    }

```

```

    printf("Optimal Page faults = %d\n",
           fault);

```

```

int main()
{
    int pages[20], n, f, i;
    printf("Enter number of pages: ");
    scanf("%d", &n);
    printf("Enter page reference string: ");
    for(i=0; i<n; i++)
        scanf("%d", &pages[i]);
    printf("Enter no of frames: ");

```

```
scanf("%d", &f);
fifo(pages, n, f);
lru(pages, n, f);
optimal(pages, n, f);
return 0;
```

y.

Output:-

Enter number of pages: 12

Enter page reference string:

1 4 0 2 6 1 2 7 3 0 2 4

Enter number of frames: 3

FIFO page faults = 11

LRU Page faults = 11

Optimal page faults = 9

~~2/1/26~~